

Student

Total Points  
14.5 / 30 pts

Question 1  
Basic questions & declarative programming

0 / 5 pts  
0 / 3 pts

1.1 Fold function

- 0 pts Correct
- 0.5 pts Small error in Fold, definition
- 1 pt Fold, definition errors
- 2 pts Fold, definition missing
- 0.5 pts Tail recursion wrong
- 0.5 pts Contextual environment wrong

✓ - 2 pts Incorrect

1.2 Invariants

- 0 pts Correct
- 1 pt Partly correct
- 2 pts Incorrect

0 / 2 pts

Question 2

Hamming problem

0 / 5 pts

- 0 pts Correct
- 0.5 pts Small error in Hamming program
- 1 pt Errors in Hamming program
- 1.5 pts Hamming program mostly wrong
- 2 pts Incorrect or missing Hamming program
- 0.5 pts Small error in Times translation
- 1 pt Errors in Times translation
- 2 pts Incorrect or missing Times translation
- 0.5 pts Incomplete explanation of WallHeaded
- 1 pt Incorrect or missing WallHeaded explanation

✓ - 3 pts Incorrect

Question 3

Limitation of declarative programming

2.5 / 5 pts

- 0 pts Correct
- 1 pt Limitation incomplete
- 2 pts Limitation wrong or missing
- 0.5 pts One extension missing
- 1 pt Extensions incomplete
- 2 pts Extensions incorrect or missing
- 1 pt Missing needed argument
- 5 pts Incorrect

Question 4

Erlich's generic server

5 / 5 pts

- 1 pt Afx incomplete
- 2 pts Afx wrong or missing
- 1 pt Functionalities incomplete
- 2 pts Functionalities wrong or missing
- 0.5 pts Callback incomplete
- 1 pt Callback wrong or missing
- 5 pts Incorrect

Question 5

Active objects and passive objects

4 / 5 pts

- 0 pts Correct
- 0.5 pts Small error in NewActive definition
- 1 pt NewActive definition incomplete
- 2 pts NewActive definition wrong or missing
- 0.5 pts Small error in passive object explanation
- 1 pt Passive object explanation incomplete
- 2 pts Passive object explanation wrong or missing
- 0.5 pts Active object explanation incomplete
- 1 pt Active object explanation wrong or missing
- 5 pts Incorrect

Question 6

Transaction manager

3 / 5 pts

- 0 pts Correct
- 1 pt Protocol incomplete
- 2 pts Protocol wrong or missing
- 0.5 pts Serializability explanation incomplete
- 1 pt Serializability explanation wrong or missing
- 0.5 pts Deadlock avoidance explanation incomplete
- 1 pt Deadlock avoidance explanation wrong or missing
- 1 pt State diagram incomplete
- 2 pts State diagram wrong or missing
- 5 pts Incorrect

# Final exam LINFO1131

January 23, 2025  
Prof. Peter Van Roy

First and last name: [REDACTED]

NOMA: [REDACTED]

*Please write all answers inside the rectangle*

**Q1: Basic concepts & declarative programming [10pts] (Q1 corresponds to the midterm)**

**Q1.1: Fold function [3pts].** Give the Oz code for the  $\{\text{FoldL L F U}\}$  function. Is your definition tail-recursive? What is the contextual environment of your definition?

**Q1.2: Invariants [2pts].** Give an invariant for an execution of the  $\{\text{FoldL L F U}\}$  function of Q1.1. Assume the list  $L = [a_0 a_1 \dots a_{n-1}]$ . Hint: the invariant shows an intermediate state of the execution of the function  $\text{fold}(l, f, u)$ . How does the principle of communicating vases work for your invariant?

First and last name

NOMA

**Q1.3: Hamming problem [5pts].**

Answer the following three questions:

1. Given the set of numbers of the form  $2^a 3^b 5^c$  with integers  $a, b, c \geq 0$ . Explain how to generate a lazy stream that gives all these numbers in strictly increasing order. Give the Oz code for a program that does this. You need to define only two functions, Times and Merge.
2. Give the kernel language translation of the function Times.
3. Your kernel language translation should use the operation WaitNeeded. Explain what the WaitNeeded operation does (give its semantics).

First and last name

NOMA

## Q2: Limitation of declarative programming [5pts]

Answer the following three questions:

1. First, give a diagram showing how to model any declarative execution as a lambda reduction. Using this reduction, explain what is the fundamental limitation of declarative programming.
2. In the course we saw three ways that declarative programming can be extended to remove this limitation. Briefly show each of these extensions in terms of an extended lambda reduction. For each extension, explain the operation that is added and show how the extension removes the limitation.
3. Given the advantages of declarative programming, and given its fundamental limitation, the course proposes what it calls "the right way to design programs". Explain what is the approach proposed by the course. Do you agree with it? Why or why not? Give a reasoned argument. Any kind of reasoning that you give will be rewarded; you do not necessarily have to agree as long as your reasoning is valid.

1.

Une des limitations fondamentales c'est le single assignment.  
on ne peut associer qu'une valeur à une variable une fois et elle ne peut pas être changée.

2.

a) les Ports  $\rightarrow$  permet de créer des listes infinies  
{New Ports}; {Send EP}

b) les cellules  $\rightarrow$  Permet de créer des variables qui ne suivent pas  
"single Assignment"  
{newcell}; @cell

c)

3. Je ne me souviens plus du nom de la façon donnée par le cours  
pour programmer mais je suis d'accord avec celle-ci elle permet  
de construire des programmes concis et optimisés.

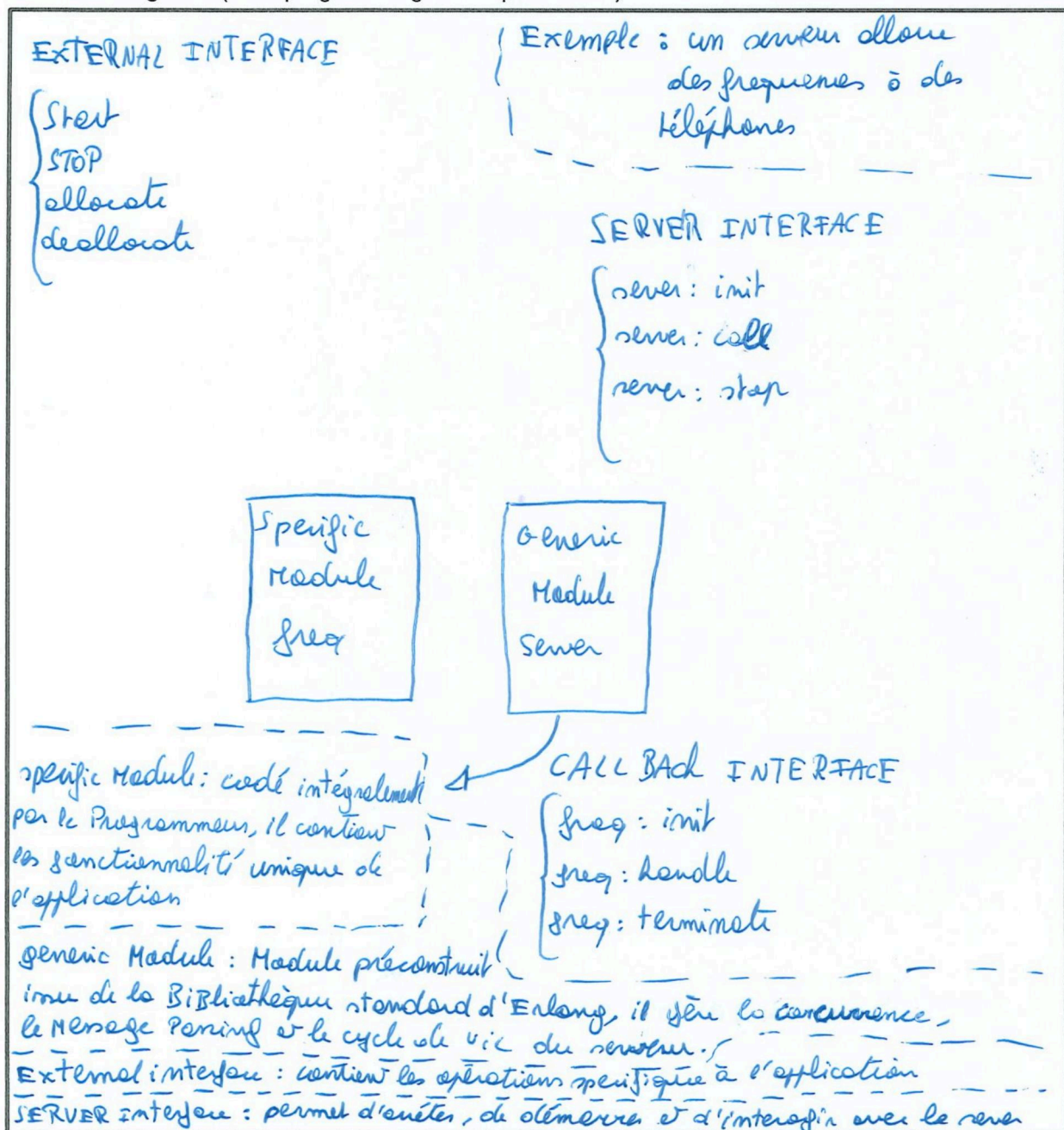


First and last name

NOM

### Q3: Erlang's generic server [5pts]

To make it easier to build concurrent and fault-tolerant applications, Erlang provides a set of generic components called "behaviors". One of the most-used Erlang behaviors is the generic server. In an application using a generic server, there will be two modules, a specific module and a generic module. For this question, give a simple example of an application that uses the generic server (show it as a block diagram). Answer the following questions. What is the functionality of the specific module (what are its operations)? What is the functionality of the generic module (what are its operations)? What does the external interface of the specific module look like? What does the external interface of the generic module look like? How are the specific and generic modules connected together (what programming technique is used)?



Le Module spécifique et générique sont connectés grâce au Process Linking 4

First and last name

NOMA

#### Q4: Active objects and passive objects [5pts]

For this question, answer the following three items:

1. Give the Oz code definition of a function NewActive that takes as arguments a class and an initial message and returns an active object. You can assume the function New exists such that  $Obj = \{New\ Class\ Init\}$  where  $Obj$  is a newly created passive object.
2. Explain why passive objects are unsafe when called from more than one thread. Use a diagram to show the precise sequence of operations in time, showing in which thread each operation occurs. Give an example to show how this can give an error.
3. Explain why active objects are safe when called from more than one thread. Use a diagram to show the precise sequence of operations in time, showing in which thread each operation occurs. Show the difference between passive and active objects that explains the active object's safety.

```

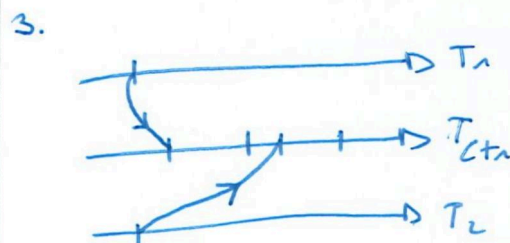
1.
fun {NewActive class Init}
  Obj = {New class Init}
  proc {Loop S}
    case S of M I T then
      {C Obj}
    end
  end
end
P

```

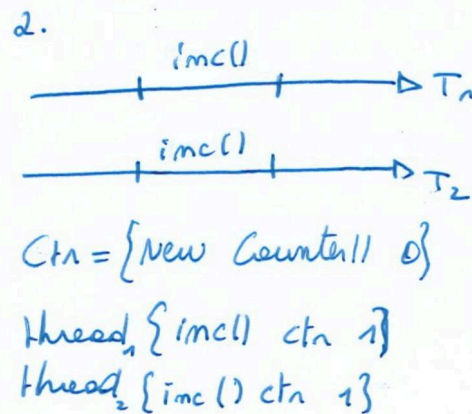
```

in
  Thread S in P = {NewPort S} {Loop S} end
P
end

```



Les Active Object sont safe car l'exécution des différents threads se fait séquentiellement sur le thread principal. Il n'y a donc aucun risque.



Si le Scheduler exécute  $T_1$  mais s'arrête après avoir @ctn et avant avoir @ctn = @ctn + 1, et que le Scheduler exécute totalement  $T_2$ , quand il va revenir exécuter  $T_1$ , il aura une mauvaise valeur à incrémenter.



First and last name: \_\_\_\_\_

NOMA: \_\_\_\_\_

### Q5. Transaction manager [5pts]

For this question answer the following two points:

1. Give a pseudocode definition of a transaction manager that is serializable and that implements deadlock avoidance. First, explain how your definition guarantees serializability. Second, explain how your definition avoids deadlocks. [3pts]
2. Give a state diagram for the transaction manager. You can give either the state diagram for the first algorithm or for the improved algorithm. [2pts]

1.

```

1.  fun {get lock}
    if {list of lock is empty} then
        {Add Lock to list}
    else if {locked transaction priority < transactionNew.priority} then
        {Restart locked transaction}
        {Get Lock transactionNew}
    else
        wait;
    end
end
  
```

Mon Pseudocode garantit la serializability grace en Taken limitant le nombre de thread qui execute la section critique

Mon Pseudocode evite les deadlocks grace au concept de "transaction priority". c'est à dire que chaque transaction est assigné à une priorité en fonction de si c'est une vieille transaction ou une nouvelle. Si une high priority transaction demande pour un lock et que l'actuelle transaction possède le lock à une plus petite priorité, cette transaction est restart et la high-priority transaction récupère le lock

2.

